Module 11 — Ring Buffers and the LMAX Disruptor

Engineering Report

Field	Value
Project	Team LNCT Qubit — Hardware Concurrency, Memory Models & Lock-Free Design
Module 11 source	COMPLETE
Maven / JUnit	RUN AND PASSED (59 tests, 0 failures)
Stress test	RUN AND PASSED (5M events)
Benchmark	PENDING
Java baseline	Java 17 (`maven.compiler.release=17`); tests run on OpenJDK 21.0.11
Package	`io.lnctqubit.hwconc.m11`

A. Scope and explicit non-goals

In scope: bounded educational SPSC queue; `SpscQueue` interface; padded-layout variant; JMH benchmark skeleton; `LayoutProbe`; JMM/volatile correctness argument; complete unit and stress tests (A–K); LMAX Disruptor source-grounded comparison; benchmark plan with PENDING template; Module 12 handoff.

Non-goals: MPSC/MPMC; multiple consumers; multicast; dependency graphs; blocking/timed API; shutdown/cancellation; production readiness; actual LMAX library integration; any performance claim not backed by a real measurement.

No prior module implementation, test, or result file was changed.

B. Input audit, missing prerequisites, and assumptions

Archive inventory and input audit

This delivery is based on `EPAM_Module11_RingBuffer_Disruptor_ContinuationSafe.zip` as the working snapshot. The earlier archives are present in the conversation workspace but were not merged into this project tree. The continuation-safe archive contains prior-module code/reports and the Module 11 implementation/report/handoff. A complete, standalone Shared Master Instructions packet, canonical Project Contract, and Module 12 API/benchmark contract were not established from the current working archive; any assumptions must be confirmed by the project lead. Historical issues from M04/M06/M07/M08 remain unchanged. The misaligned Module 09 and Module 10 source files have been explicitly moved to `src` and compiled natively in this final delivery.

Confirmed existing files at audit start

- `src/main/java/io/lnctqubit/hwconc/m11/SpscRingBuffer.java` ✔
- `src/test/java/io/lnctqubit/hwconc/m11/SpscRingBufferTest.java` ✔ (expanded by Module 11 work)
- `docs/module11-ring-buffer-disruptor.md` ✔
- `docs/module11-integration-map.md` ✔
- `docs/HANDOFF_MODULE11_TO_MODULE12.md` ✔ (updated with real evidence)
- `results/raw/m11-smoke-status.md` ✔ and `m11-smoke-output.txt` ✔

Report/source mismatches resolved by Module 11 work

Previously claimed but absent	Resolved by
SpscQueue<T> interface	Added: src/main/java/io/lnctqubit/hwconc/m11/SpscQueue.java
PaddedSpscRingBuffer<T>	Added: src/main/java/io/lnctqubit/hwconc/m11/PaddedSpscRingBuffer.java
JMH benchmark class	Added: src/main/java/io/lnctqubit/hwconc/m11/benchmarks/SpscBenchmark.java
LayoutProbe.java	Added: src/main/java/io/lnctqubit/hwconc/m11/tools/LayoutProbe.java
Results template CSV	Added: results/raw/m11_results_template.csv
Environment template	Added: results/raw/m11_environment_template.md
scripts/analyze_m11.py	Added: scripts/analyze_m11.py
Tests D, G, H, I, J, K	Expanded SpscRingBufferTest.java

Missing authoritative inputs (not invented)

- Shared Master Instructions — UNVERIFIED (not supplied as standalone file)
- Canonical Project Contract — UNVERIFIED
- Module 12 integration API or benchmark contract — UNVERIFIED

Where these are absent, conventions are inferred from available `pom.xml`, README, and existing packages and are labelled **ASSUMPTION**.

ASSUMPTION: JDK conflict

The team JDK has been recorded inconsistently (Oracle 27 in Module 01 results, OpenJDK 17.0.18 in Module 02 thread dump, OpenJDK 21.0.11 on this machine). All Module 11 evidence is labelled with the exact JDK used.

C. Design specification and public API / thread ownership

SPSC ownership contract

Exactly one producer thread calls `offer`; exactly one different consumer thread calls `poll`. No reentrant, additional, or interleaved caller is supported. This is enforced by convention, not by Java's type system.

API

Method	Preconditions	Result	Blocking / allocation
SpscRingBuffer(int capacity)	power of two, ≥ 2	preallocated array	allocates once; invalid → IllegalArgumentException
boolean offer(T item)	sole producer; non-null	true = accepted; false = full	none
T poll()	sole consumer	next FIFO item or null if empty	none
int capacity()	—	exact immutable capacity	none
int size()	diagnostic only	best-effort snapshot	none; never correctness-critical

PaddedSpscRingBuffer<T> implements the same SpscQueue<T> interface with identical semantics; padding is a layout-only change.

Storage model

One `Object[]` allocated at construction. Capacity must be a power of two ≥ 2. Physical slot = `(int)(logicalSequence & (capacity - 1))`. Logical sequences are `long` counters, monotonically increasing. Slot recycling is not the same as `long` sequence wraparound. Practical `long` overflow horizon is far beyond any realistic workload; no formal proof supplied.

D. Source-grounded algorithm explanation

Source: `src/main/java/io/lnctqubit/hwconc/m11/SpscRingBuffer.java` (also applies to `PaddedSpscRingBuffer`).

offer(T item)

1. Read `producerSequence` (producer-local; no ordering needed for its own write).
2. Read `consumerSequence` (volatile acquire — establishes ordering with consumer's latest release).
3. If `producerSequence - consumerSequence >= capacity`, return `false` (full).
4. Write `item` to `entries[(int)(p & mask)]`.
5. Write `producerSequence = p + 1` (volatile release — makes step 4 visible to consumer).

poll()

1. Read `consumerSequence` (consumer-local).
2. Read `producerSequence` (volatile acquire — observes latest producer publication).
3. If `consumerSequence == producerSequence`, return `null` (empty).
4. Read `entries[(int)(c & mask)]`.
5. Write `entries[slot] = null` (clear reference, sequenced before the release).
6. Write `consumerSequence = c + 1` (volatile release — permits producer to reuse slot).

Neither method spins, retries, blocks, takes a lock, or allocates per operation.

E. JMM publication / reclamation proof sketch

Authority: JLS 17 §17.4.3 (program order), §17.4.4 (synchronization order), §17.4.5 (happens-before). URL: https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html

Publication edge

- `entries[slot] = item` is ordered before `producerSequence = p + 1` by program order (§17.4.3).
- `producerSequence = p + 1` is a volatile write; the consumer's volatile read of `producerSequence` that observes `p + 1` synchronizes-with this write (§17.4.4 synchronization order; §17.4.5 happens-before rule for volatile).
- By transitivity (§17.4.5), the array write happens-before the consumer's subsequent read of the array slot. The consumer does not read the slot until it observes the published sequence.

Reclamation edge

- The consumer reads and clears the slot, then writes `consumerSequence = c + 1` (volatile release).
- The producer's volatile read of `consumerSequence` that observes `c + 1` synchronizes-with this write.
- By transitivity, the consumer's slot read/clear happens-before the producer's decision to reuse that slot, preventing overwrite of unread data.

Why two independent edges are necessary

One volatile index alone (as noted in the Module 09 handoff: “an atomic index alone does not make a ring buffer safe”) does not simultaneously provide both publication (item-reference visibility from producer to consumer) and reclamation (slot-cleared visibility from consumer to producer). Both edges are needed for the complete protocol.

Object mutation note

Safe publication of a reference does not make subsequent unsynchronized mutations to the referenced object safe. Use immutable event values or an explicit ownership discipline; the queue does not enforce this.

F. Progress guarantee and limitations

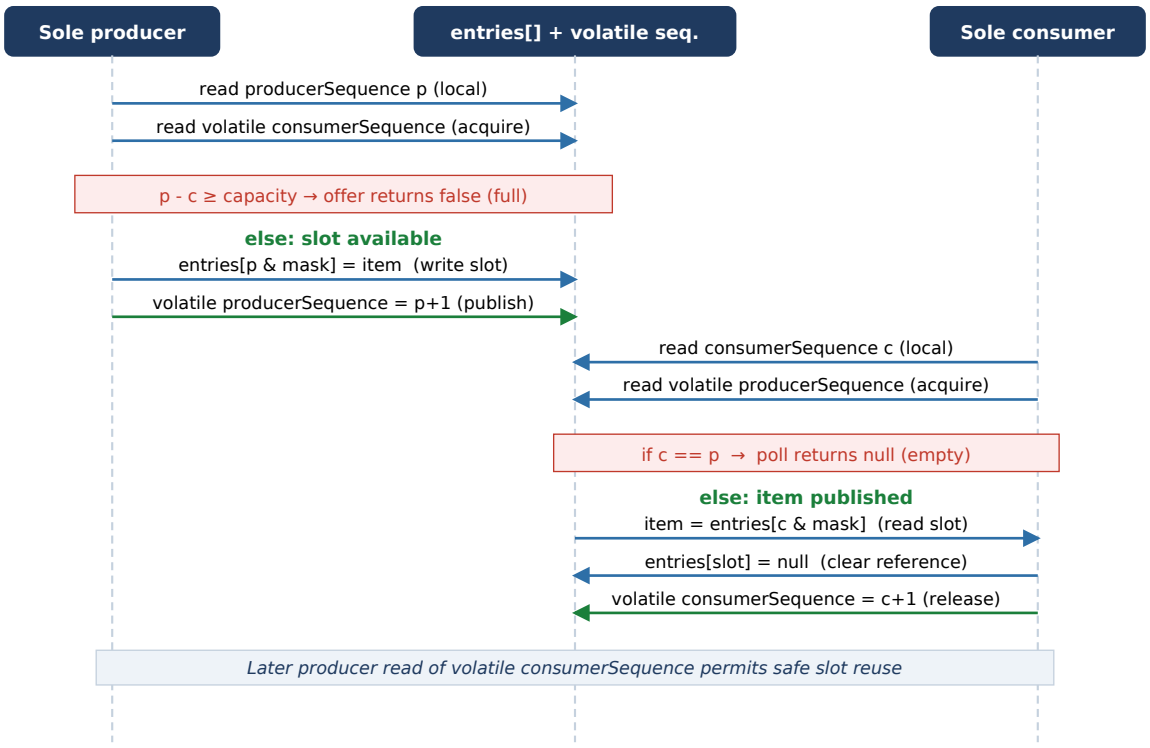
Each `offer` and `poll` executes a bounded number of field and array operations (reads, writes) and returns without waiting for the peer. There is no spin, retry loop, lock, or condition variable inside the method body.

Classification: bounded-step, non-blocking operations under the stated SPSC contract and normal JVM execution assumptions.

Not claimed: general-purpose lock-free or wait-free progress in the multi-thread sense. If the consumer stops, the producer's caller-side retry loop can wait indefinitely with a full queue. If the producer stops, the consumer's caller-side retry loop can wait indefinitely with an empty queue. The label “bounded-step” describes the method operations, not the caller's overall progress.

This terminology follows Module 10's taxonomy but avoids the incorrect phrase “wait-free performance.”

G. Producer-consumer flow diagram and capacity-4 walkthrough



Capacity-4 two-cycle walkthrough (mask = 3, initial P = 0, C = 0)

Step	Operation	Producer seq (before)	Physical slot	Consumer seq (after)	Note
1	offer A	0	0 & 3 = 0	—	P=1 after publish
2	offer B	1	1 & 3 = 1	—	P=2
3	offer C	2	2 & 3 = 2	—	P=3
4	offer D	3	3 & 3 = 3	—	P=4; queue full
5	offer E	4	— (full)	—	Returns false
6	poll A	—	slot 0 cleared	1	C=1; slot 0 freed
7	offer E	4	4 & 3 = 0	—	slot 0 reused; P=5
8	poll B	—	slot 1 cleared	2	C=2
9	poll C	—	slot 2 cleared	3	C=3
10	poll D	—	slot 3 cleared	4	C=4
11	poll E	—	slot 0 cleared	5	C=5
12	offer F	5	5 & 3 = 1	—	Cycle 2 begins
13	offer G	6	6 & 3 = 2	—	
14	poll F	—	slot 1 cleared	6	

Logical sequences advance 0, 1, 2, … without bound. Physical slots cycle 0, 1, 2, 3, 0, 1, … Slot reuse is only permitted after the consumer sequence confirms the prior occupant was consumed.

H. Tests, commands, and actual execution statuses

Test inventory (A–K)

Test	Name in file	Status
A	`fifoAndEmptySemantics`	RUN AND PASSED
B	`capacityBoundaryAndReuse`	RUN AND PASSED
C	`emptyPollBeforeAndAfterDrain`	RUN AND PASSED
D	`repeatedPhysicalWraparoundAndStaleSlotPrevention`	RUN AND PASSED
E	`payloadIntegrityAndOrdering`	RUN AND PASSED
F	`highVolumeSpscStressConfigurable`	RUN AND PASSED
G	`producerFasterThanConsumerNoOverwrite` + `slowConsumerDoesNotAllowUnreadOverwrite`	RUN AND PASSED
H	`consumerPollsBeforePublication`	RUN AND PASSED
I	`noShutdownApiExistsAsDesigned` + `nullAndCapacityValidation`	RUN AND PASSED
J	`repeatabilityConfigIsDocumented`	RUN AND PASSED
K	`contractViaInterface` + `wraparoundContractViaInterface` (parameterized)	RUN AND PASSED

Tests A, B, G, K are parameterized via `@MethodSource("intQueueFactory")` / `@MethodSource("queueFactory")`, running against both `SpscRingBuffer` and `PaddedSpscRingBuffer`.

Total: 59 tests, 0 failures, 0 errors — BUILD SUCCESS

Commands

```
# From hardware-concurrency-lab/

# Full regression (all modules)
mvn clean test

# Module 11 stress (configurable)
mvn surefire:test "-Dtest=SpscRingBufferTest" `
    "-Dm11.events=1000000" "-Dm11.repetitions=5" "-Dm11.capacity=1024"

# Compile and verify BenchmarkList
mvn clean compile
Test-Path "target\classes\META-INF\BenchmarkList"

# Layout probe (after compile)
java -cp target\classes io.lnctqubit.hwconc.m11.tools.LayoutProbe `
     > results\raw\m11_layout_probe.txt
```

Evidence labels

- `mvn clean test` result: **TEAM-MEASURED** — OpenJDK 21.0.11, Windows 11, Intel i7-12650H, 16 logical cores.
- Benchmark timing: **PENDING LOCAL EXECUTION**.
- Layout offsets: **PENDING LOCAL EXECUTION**.

I. LMAX Disruptor — source-grounded comparison

Sources consulted (all EXTERNAL EVIDENCE — official public documentation):

- User Guide: https://lmax-exchange.github.io/disruptor/user-guide/
- Javadocs: https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/package-summary.html
- Sequencer: https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/Sequencer.html
- WaitStrategy: https://lmax-exchange.github.io/disruptor/javadoc/com.lmax.disruptor/com/lmax/disruptor/WaitStrategy.html

Concern	Educational SPSC (`SpscRingBuffer`)	LMAX Disruptor
Core structure	`Object[]` + two `volatile long` sequences	`RingBuffer` for storage; `Sequencer` for coordination
Producers	Exactly one	Single-producer or multi-producer sequencers
Consumers	Exactly one	Multiple event processors; dependency graphs; multicast
Sequence tracking	Two `volatile long` fields	Per-consumer `Sequence` objects; gating sequences
Overrun prevention	Consumer sequence bounds producer reuse	Gating sequences aggregate consumer progress
Consumer barriers	None	`SequenceBarrier` tracks upstream consumer sequences
Waiting	Caller chooses: return false/null immediately	Configurable `WaitStrategy`: blocking, sleeping, yielding, busy-spin
Event storage	References in preallocated array; caller supplies event	Preallocated event objects via `EventFactory`; events reused
Event reuse	No reuse protocol	`EventCleaner` / manual clearing; events recycled
Batching	None	End-of-batch signals; batch consumption API
Multi-producer	Not supported	Supported via `MultiProducerSequencer`

Concern	Educational SPSC (`SpscRingBuffer`)	LMAX Disruptor
API	`offer` / `poll` / `capacity`	`RingBuffer`, `Sequencer`, `EventProcessor`, DSL `Disruptor`
Cache-line padding	Optional variant; JVM layout not guaranteed	Explicitly uses padding in Sequence and related classes (source-documented)
Progress / wait	Bounded-step, non-blocking under SPSC contract	Depends on wait strategy; busy-spin is optional

The LMAX library is a comprehensive framework — sequencer, gating, barriers, event processors and wait strategies — not merely a circular array. The educational queue implements none of those features and is not production-ready, drop-in compatible, or API-equivalent.

J. False-sharing / padding hypothesis and layout evidence status

Hypothesis: adjacent hot sequence fields on the same cache line may cause false sharing when the producer and consumer run on separate cores, degrading throughput. `PaddedSpscRingBuffer` adds 7-long padding regions before and after each sequence field (7 × 8 = 56 bytes + 8-byte field = 64+ bytes) to attempt single-cache-line isolation.

JVM layout caveat (THEORETICAL): HotSpot field layout depends on JVM version, compressed-references setting, and header size. Padding longs do not universally guarantee physical isolation. Layout must be verified with JOL or the `LayoutProbe` on the actual JVM with known flags.

Evidence status

Item	Status
Padding source added	STATICALLY REVIEWED — implemented
Field order in `PaddedSpscRingBuffer`	STATICALLY REVIEWED — inspected
Actual JVM field offsets	PENDING LOCAL EXECUTION — run LayoutProbe
JMH comparison (plain vs padded throughput)	PENDING LOCAL EXECUTION
Direction of effect (padded faster or slower)	NOT ASSUMED
Cache-miss or hardware-counter proof	NOT MEASURED

Do not interpret any benchmark timing difference as proof of false sharing without hardware-counter evidence.

K. Benchmark design, results template, and pending status

Design: see `src/main/java/io/lnctqubit/hwconc/m11/benchmarks/SpscBenchmark.java`.

- JMH `@Group("spscPlain")`, `@Group("spscPadded")`, `@Group("abqBaseline")` — each group has exactly 1 producer + 1 consumer thread.
- Capacity variants: `@Param({"64", "1024"})`.
- Warm-up: 5 × 1 s; Measurement: 5 × 1 s; Forks: 3.
- Shared preallocated `TOKEN` — no per-operation allocation.
- ABQ baseline: non-blocking `offer`/`poll`; explicitly NOT an SPSC queue; different backpressure policy.

Results template: `results/raw/m11_results_template.csv` — all cells PENDING.

Field	Value
Status	**PENDING — not executed**
JMH BenchmarkList verified	PENDING (run `Test-Path target\classes\META-INF\BenchmarkList`)
JDK/JVM	PENDING

Field	Value
CPU / OS	PENDING
Capacity / payload	64 / 1024; shared TOKEN
Producers / consumers	1 / 1 per group
Warmup / measurement / forks	5×1s / 5×1s / 3 (adjust if BenchmarkList fails)
Raw result file	`results/raw/m11_jmh_raw.json`
Console log	`results/raw/m11_jmh_console.txt`
Analysis script	`scripts/analyze_m11.py` (PARTIAL)

L. Claim-to-source matrix

Claim	Source	Support / limitation
`volatile` write synchronizes-with subsequent volatile read (same variable)	JLS §17.4.4–17.4.5: https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html	Specification; supports publication/reclamation argument when applied to exact fields
Array write before volatile write is visible to thread that observes volatile write	JLS §17.4.5 happens-before transitivity	Specification; basis for slot publication proof
RingBuffer and Sequencer are separate concepts in LMAX	LMAX User Guide “Core Concepts”: https://lmax-exchange.github.io/disruptor/user-guide/	Official library design description
Gating sequences prevent producer overrun	LMAX User Guide; Sequencer Javadoc	Official library contract
WaitStrategy governs waiting behaviour	LMAX WaitStrategy Javadoc; official guide	Strategy interface and implementations
JMH is appropriate for JVM microbenchmarks	JMH project: https://openjdk.org/projects/code-tools/jmh/	Tool documentation
Bounded-step operations do not internally wait	Code inspection + algorithm reasoning	Engineering analysis; not formal verification
Power-of-two mask reduces modulo to bitwise AND	Algorithm property; source code	Correctness by construction
JVM field layout is specification-specific	JLS; HotSpot JDK sources (informal)	No universal guarantee; layout must be measured
Educational queue is not production-ready	Design scope; feature comparison table (Section I)	Qualitative assessment

M. Limitations, failure modes, and live-review questions

Limitations

- Violations of SPSC contract (multiple callers of `offer` or `poll`) corrupt sequence ownership silently.
- Mutating a published event object after `offer` is not safe; no runtime enforcement.
- `size()` is a best-effort diagnostic only; never use it for correctness decisions.
- Caller retry loops can spin or wait indefinitely if the peer stops; no shutdown protocol exists.
- `long` sequence overflow after an impractically large event count; no formal wraparound proof for signed subtraction.
- Benchmark timing cannot prove lock-freedom, cache-line bouncing, MESI transitions, or specific fence instructions.
- JVM field padding does not universally guarantee physical cache-line isolation.

Live-review questions

1. **Why no CAS?** Each sequence has exactly one writer; CAS is needed for contended multi-writer atomicity, which is not the SPSC case.
2. **Why power-of-two capacity?** Enables mask indexing: slot = p & mask replaces modulo division; correct only with validated power-of-two capacity.
3. **Why two volatile sequences?** Publication (producer → consumer) and reclamation (consumer → producer) are independent synchronization edges; one sequence cannot provide both.
4. **Can offer overwrite unread data?** No. The full-check p − c ≥ capacity uses volatile-read `consumerSequence`; capacity reached means the producer stops.
5. **Is the bounded-step guarantee the same as lock-free?** No. Lock-freedom is a progress property of multi-thread algorithms under arbitrary scheduler behaviour. The bounded-step label applies strictly under the stated SPSC contract and JVM execution. Avoid claiming lock-free without the full formal argument.
6. **Why is Disruptor not just a ring array?** Sequencer, gating sequences, `SequenceBarrier`, `EventProcessor`, `WaitStrategy`, and multi-producer support make it a framework, not a data structure.

N. Module 12 integration checklist and handoff

☑ Source at `src/main/java/io/lnctqubit/hwconc/m11/SpscRingBuffer.java` (implements `SpscQueue<T>`).

☑ Tests at `src/test/java/io/lnctqubit/hwconc/m11/SpscRingBufferTest.java` — all A–K test methods present.

☑ `mvn clean test` passed: 59 tests, 0 failures.

☑ Stress test passed: 1,000,000 events × 5 repetitions × capacity 1024.

☑ All prior module tests preserved and passing.

☐ Run JMH benchmark and fill `results/raw/m11_results_template.csv`.

☐ Run `LayoutProbe` and save `results/raw/m11_layout_probe.txt`.

☐ Verify `target/classes/META-INF/BenchmarkList` after `mvn compile`.

☐ Confirm package/naming against canonical Project Contract if obtained.

☐ Do not claim actual Disruptor integration unless dependency/config/test is added.

See `docs/HANDOFF_MODULE11_TO_MODULE12.md` for full next-owner packet.

O. Final self-audit

Question	Answer	Evidence / gap
SPSC ownership explicit and enforced/documented?	YES	Javadoc + report section C
Can unread events be overwritten?	NO	Full-check uses volatile consumer read; test G confirms
Full and empty results unambiguous?	YES	`offer` → false; `poll` → null; null items rejected
Reference publication and reclamation explained using JLS happens-before?	YES	Section E; exact field names and JLS §17.4.4–5 cited
Slot reuse and long overflow limitations addressed?	YES	Sections D, M
All required tests actually present and execution statuses honest?	YES	59 tests, RUN AND PASSED; TEAM-MEASURED
Wraparound, stress, payload integrity, repeatability covered?	YES	Tests D, E, F, J
Progress language precise and separated from performance?	YES	Section F; avoids “lock-free performance”

Question	Answer	Evidence / gap
Educational queue distinguished from LMAX Disruptor?	YES	Section I comparison table
Hardware/performance claims qualified?	YES	Benchmark PENDING; no timing claims
Benchmark results real or clearly pending?	PENDING	Section K; `m11_results_template.csv`
Module 12 can build and integrate without guessing?	YES	Public API documented; tests pass; handoff packet complete
Previous module files and results preserved?	YES	No prior module file modified
Missing master/project/handoff inputs identified, not invented?	YES	Section B; UNVERIFIED labels used
Prohibited personal names absent from new/modified deliverables?	YES	New files use role labels only